2025 WINTER SEMESTER
Programming for Data Science (BCSE207P) Lab Record
Practice Program(PP)/ Exercise Program(EP)

Date:10/3/26 **Week No.:** **Program S. No. 35**

Title	Course Outcome Analytics with CSV I/O (Base R Only)
Problem Statement	

- Construct a student marks data frame from a given table using **column vectors**.
- **Persist** the data to CSV with `write.csv()` and **reload** it with `read.csv()`.
- Compute per-student **Total**, **Average**, and **Rank** (highest total = rank 1).
- Compute **assessment-wise averages** across the class.
- Compute **specialization-wise averages** (per assessment and overall).

Save all analytics outputs to **separate CSV files** (base R only; no packages).

use these values **exactly** to construct vectors and the initial data frame:

RegNo	Name	Programme	Specialization	Assess1	Assess2	Assess3	Assess4
24BCE1001	Aadesh Kumar	B.Tech	CSE	18	17	19	18
24BCE1002	Bhavya Reddy	B.Tech	AI	15	16	14	18
24BCE1003	Charan Iyer	B.Tech	DS	20	19	18	20
24BCE1004	Divya Sharma	B.Tech	IT	14	15	15	16
24BCE1005	Esha Nair	B.Tech	CSE	20	20	19	20
24BCE1006	Farhan Khan	B.Tech	ECE	12	14	13	16
24BCE1007	Gayathri Raj	B.Tech	Cyber	17	18	17	19
24BCE1008	Harish Kumar	B.Tech	AI	16	17	16	15
24BCE1009	Ishita Menon	B.Tech	DS	19	18	19	18
24BCE1010	Jai Verma	B.Tech	IT	18	17	16	19

- **Input file**: `students_input.csv`
- **Output files**:
  - `students_by_rank.csv`
  - `assessment_wise_averages.csv`
  - `specialization_wise_averages.csv`

(Optional: store in an `output_csv/` folder.)

Tasks

Task 1 — Create Data Frame from the Table

1. Create vectors for: `RegNo`, `Name`, `Programme`, `Specialization`, `Assess1`, `Assess2`, `Assess3`, `Assess4`.
2. Combine into data frame **`students`** with `stringsAsFactors = FALSE`.
3. Print `str(students)` and `head(students)`.

Task 2 — Write Input CSV

- Save `students` to **`students_input.csv`** using `write.csv(..., row.names = FALSE, na = "")`.

Task 3 — Read from CSV for Analytics

1. Read **`students_input.csv`** into **`students_in`** with `read.csv(stringsAsFactors = FALSE)`.
2. Validate required columns exist.
3. Coerce assessment columns (`Assess1:Assess4`) to numeric if needed.

Task 4 — Per-Student Analytics (from CSV input)

1. Compute `Total = Assess1 + Assess2 + Assess3 + Assess4`.
2. Compute `Average = Total / 4`.
3. Compute `Rank` where **highest Total = 1** (e.g., `rank(-Total, ties.method="min")`).
4. Sort ascending by `Rank` (and by `RegNo` to stabilize ties).
5. Write the sorted result to **`students_by_rank.csv`**.

Task 5 — Assessment-Wise Averages (overall)

1. Use `colMeans()` on the numeric columns `Assess1:Assess4` (`na.rm=TRUE`).
2. Convert the named vector to a 2-column data frame with headers: `Assessment`, `Average`.
3. Write to **`assessment_wise_averages.csv`**.

Task 6 — Specialization-Wise Averages (per assessment + overall)

1. Use `aggregate()` with `by = list(Specialization = students_in$Specialization)` over `Assess1:Assess4` (mean, `na.rm=TRUE`).
2. Add `Overall_Average = rowMeans(...)` across Assess1–Assess4.
3. Optionally sort by `Overall_Average` (descending).
4. Write to **`specialization_wise_averages.csv`**.

Note: Stick to base R; use vectorized ops (`rowSums`, `colMeans`, `aggregate`); avoid loops.

Program

```
## ------------------------------------------------------------
```

```
## Course Outcome Analytics with CSV I/O (Base R Only)
## ------------------------------------------------------------

## 0. File names
input_csv <- "students_input.csv"
rank_csv <- "students_by_rank.csv"
assessment_avg_csv <- "assessment_wise_averages.csv"
spec_avg_csv <- "specialization_wise_averages.csv"


## ------------------------------------------------------------
## Task 1 — Create Data Frame from Table
## ------------------------------------------------------------

# Create vectors from the dataset

RegNo <-
c("24BCE1001","24BCE1002","24BCE1003","24BCE1004","24BCE1005"
,

"24BCE1006","24BCE1007","24BCE1008","24BCE1009","24BCE1010")

Name <- c("Aadesh Kumar","Bhavya Reddy","Charan Iyer","Divya
Sharma","Esha Nair",
        "Farhan Khan","Gayathri Raj","Harish Kumar","Ishita Menon","Jai
Verma")

Programme <- c("B.Tech","B.Tech","B.Tech","B.Tech","B.Tech",
             "B.Tech","B.Tech","B.Tech","B.Tech","B.Tech")

Specialization <-
c("CSE","AI","DS","IT","CSE","ECE","Cyber","AI","DS","IT")

Assess1 <- c(18,15,20,14,20,12,17,16,19,18)
Assess2 <- c(17,16,19,15,20,14,18,17,18,17)
Assess3 <- c(19,14,18,15,19,13,17,16,19,16)
Assess4 <- c(18,18,20,16,20,16,19,15,18,19)


# Combine vectors into a data frame

students <- data.frame(
  RegNo,
  Name,
  Programme,
  Specialization,
  Assess1,
  Assess2,
```

```
  Assess3,
  Assess4,
  stringsAsFactors = FALSE
)

# Display structure and preview
cat("Structure of students dataframe:\n")
str(students)

cat("\nFirst 6 rows of students dataframe:\n")
print(head(students))


## ------------------------------------------------------------
## Task 2 — Write Input CSV
## ------------------------------------------------------------

write.csv(students, file = input_csv, row.names = FALSE, na = "")
cat("\nInput CSV file written:", input_csv, "\n")


## ------------------------------------------------------------
## Task 3 — Read CSV for Analytics
## ------------------------------------------------------------

students_in <- read.csv(input_csv, stringsAsFactors = FALSE)

# Validate required columns
required_cols <- c("RegNo","Name","Programme","Specialization",
                   "Assess1","Assess2","Assess3","Assess4")

missing_cols <- setdiff(required_cols, names(students_in))

if(length(missing_cols) > 0){
  stop(paste("Missing columns:", paste(missing_cols, collapse=", ")))
}

# Ensure assessment columns are numeric
assessment_cols <- c("Assess1","Assess2","Assess3","Assess4")

for(col in assessment_cols){
  if(!is.numeric(students_in[[col]])){
    students_in[[col]] <- as.numeric(students_in[[col]])
  }
}


## ------------------------------------------------------------
```

```
## Task 4 — Per Student Analytics
## ------------------------------------------------------------

# Compute Total
students_in$Total <- rowSums(students_in[, assessment_cols])

# Compute Average
students_in$Average <- students_in$Total / 4

# Compute Rank (Highest Total = Rank 1)
students_in$Rank <- rank(-students_in$Total, ties.method = "min")

# Sort by Rank then RegNo
students_ranked <- students_in[order(students_in$Rank,
students_in$RegNo), ]

# Save to CSV
write.csv(students_ranked, file = rank_csv, row.names = FALSE)

cat("Student ranking saved to:", rank_csv, "\n")


## ------------------------------------------------------------
## Task 5 — Assessment Wise Averages
## ------------------------------------------------------------

assessment_avg <- colMeans(students_in[, assessment_cols], na.rm =
TRUE)

assessment_avg_df <- data.frame(
  Assessment = names(assessment_avg),
  Average = as.numeric(assessment_avg),
  stringsAsFactors = FALSE
)

write.csv(assessment_avg_df, file = assessment_avg_csv, row.names =
FALSE)

cat("Assessment averages saved to:", assessment_avg_csv, "\n")


## ------------------------------------------------------------
## Task 6 — Specialization Wise Averages
## ------------------------------------------------------------

spec_avg <- aggregate(
  students_in[, assessment_cols],
  by = list(Specialization = students_in$Specialization),
```

```
  FUN = function(x) mean(x, na.rm = TRUE)
)

# Add Overall Average
spec_avg$Overall_Average <- rowMeans(spec_avg[, assessment_cols])

# Sort by Overall Average (descending)
spec_avg <- spec_avg[order(-spec_avg$Overall_Average), ]

# Save to CSV
write.csv(spec_avg, file = spec_avg_csv, row.names = FALSE)

cat("Specialization averages saved to:", spec_avg_csv, "\n")


## ------------------------------------------------------------
## End of Program
## ------------------------------------------------------------
```

Output Snapshots

```
+
+ # Add Overall Average
+ spec_avg$Overall_Average <- rowMeans(spec_avg[, assessment_cols])
+
+ # Sort by Overall Average (descending)
+ spec_avg <- spec_avg[order(-spec_avg$Overall_Average), ]
+
+ # Save to CSV
+ write.csv(spec_avg, file = spec_avg_csv, row.names = FALSE)
+
+ cat("Specialization averages saved to:", spec_avg_csv, "\n")
+
+
+ ## ------------------------------------------------------------
+ ## End of Program
+ ## ------------------------------------------------------------
+
```

```
+ ## ------------------------------------------------------------
+ ## Task 5 — Assessment Wise Averages
+ ## ------------------------------------------------------------
+
+ assessment_avg <- colMeans(students_in[, assessment_cols], na.rm = TRUE)
+
+ assessment_avg_df <- data.frame(
+   Assessment = names(assessment_avg),
+   Average = as.numeric(assessment_avg),
+   stringsAsFactors = FALSE
+ )
+
+ write.csv(assessment_avg_df, file = assessment_avg_csv, row.names = FALSE)
+
+ cat("Assessment averages saved to:", assessment_avg_csv, "\n")
+
+
```

Explanation/ Observations

1. Creation of Vectors

- Individual vectors were created using the `c()` function for each column:
  - **RegNo**
  - **Name**
  - **Programme**
  - **Specialization**
  - **Assess1–Assess4**
- Character vectors store student information.
- Numeric vectors store assessment scores.

---

2. Creation of Data Frame

- All vectors were combined using the `data.frame()` function.
- `stringsAsFactors = FALSE` ensures text values remain character data.

- `str(students)` displays the structure of the data frame.
- `head(students)` shows the first six rows.

---

3. Writing the Input CSV

- The dataset was exported to **students_input.csv** using `write.csv()`.
- `row.names = FALSE` prevents row numbers from being saved in the file.

---

4. Reading and Validating the CSV

- The CSV file was reloaded using `read.csv()`.
- Required columns were validated to ensure dataset completeness.
- Assessment columns were verified and converted to **numeric type** if required.

---

5. Per-Student Analytics

- **Total** marks were calculated using `rowSums()`.
- **Average** marks were calculated as `Total / 4`.
- **Rank** was calculated using `rank(-Total, ties.method="min")` so the highest total receives rank 1.
- The results were sorted by **Rank and RegNo** and saved to **students_by_rank.csv**.

---

6. Assessment-Wise Averages

- The average score of each assessment across all students was calculated using `colMeans()`.
- The results were stored in a table with columns:
  - **Assessment**
  - **Average**
- Saved to **assessment_wise_averages.csv**.

---

7. Specialization-Wise Averages

- The `aggregate()` function calculated the average marks for each specialization.
- `rowMeans()` calculated an **Overall_Average** across all assessments.
- The results were sorted by **Overall_Average** in descending order.
- Saved to **specialization_wise_averages.csv**.

References